

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been copied and communicated to you by or on behalf of The University of Newcastle pursuant to Part VB of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

SENG1050

Web Technologies

Week 8 - JavaScript

Lecture Outline

- ❑ Introduction to JavaScript
- ❑ JavaScript Basics
- ❑ Form Validation
- ❑ Comments and Methods



Recall from last week: Content Management Systems

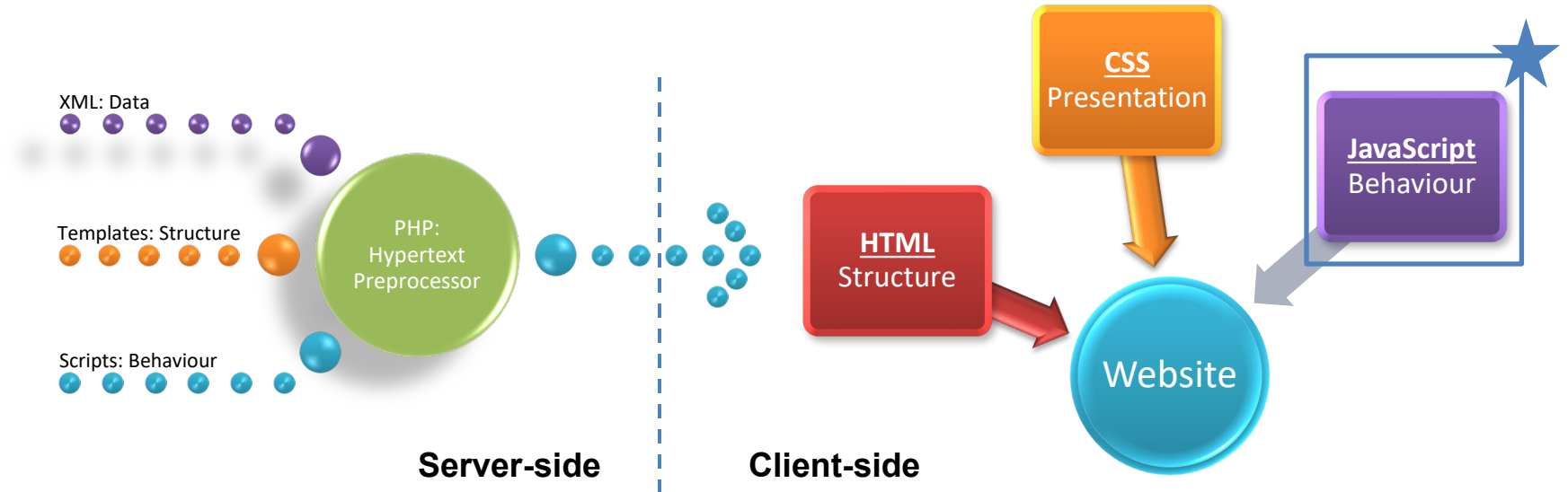
- ❑ Last week, we explored the issue of scalability, and discovered how content management systems can help organise and manage content as a website grows in size.
- ❑ We also learned how to create templates to reduce duplicated HTML code, and to pull data from XML files into HTML documents.
- ❑ This week, we will complete the separation of concerns we started in Week 3 by looking at how dynamic, client-side behaviour is implemented using JavaScript.

Introduction to JavaScript

Overview of Client-based Web Development



The Server-Based Model



Introduction

- ❑ JavaScript is the programming language of the web.
- ❑ Powerful feature: Any code written in JavaScript can run as-is in any modern web browser.
- ❑ It is the most **ubiquitous** programming language in history because it is used in all modern web browsers on
 - Desktops
 - Game consoles
 - Tablets
 - Smart phones
- ❑ Easy to learn, Developed by Netscape in 1995
- ❑ Often abbreviated as JS

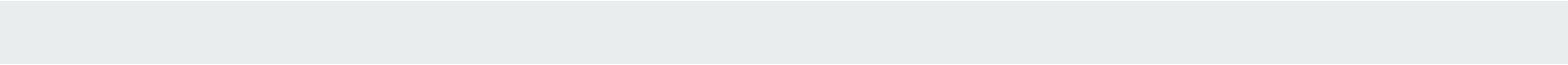
What Can JavaScript Do?

- ❑ Add **dynamic features** to Web pages
- ❑ JavaScript is executed when it is encountered as the HTML is loaded
 - Script can perform calculations
 - Script can define functions for use by other scripts on the page
 - Script can dynamically generate HTML

```
write("<h1>My Home Page</h1>")
```
- ❑ JavaScript enables shopping carts, **form validation**, special graphic and text effects, image swapping, image mapping, **dynamic interfaces**, clocks, **handling cookies** and more.

JavaScript vs. Java vs. HTML

- ❏ The name “JavaScript” is somewhat misleading
- ❏ JavaScript ≠ Java
 - It “looks” like Java, but it is not Java
 - It is *object-oriented*, just like Java
 - But, it is much simpler than Java
 - JavaScript is *(usually) interpreted*
- ❏ JavaScript ≠ HTML
 - But it was designed to work inside HTML



JavaScript vs. PHP

- ❑ Is this looking familiar?
- ❑ JavaScript and PHP look **very** similar (aside from the '\$' in front of variable names).
- ❑ PHP can also work with objects (remember the -> operator from last week?). Both are scripting languages, both are platform independent (they run on all operating systems).
- ❑ They serve different functions – JavaScript runs natively in the browser, whereas PHP is strictly run on the web server; it cannot run natively on a client. PHP is primarily used to generate HTML code. JavaScript does not run natively on a web server without additional frameworks (such as Node.js). Even servers running a JavaScript framework often use some PHP scripts as well.
- ❑ Client-side JavaScript is exposed to the client, whereas PHP is invisible, only the HTML output is returned.



Object-Oriented



An analogy

- ❑ A car is a physical **object**
- ❑ A car has certain **properties** – make, model, colour, number of doors, distance driven, age
- ❑ There are certain **actions (methods)** you can perform on the car – start, stop, turn, spray paint; **they often affect the car's properties**
- ❑ Cars can respond to certain **events** – e.g., colliding with another car

Terminology

- ❖ **Object:** A black-box that stores information
- ❖ **Property:** Directly accessible pieces of information
- ❖ **Method:** Something an object can DO – often used to access or change information
- ❖ **Event:** An actions or occurrence that affects an object – **triggers** a method



Object-Oriented and JavaScript

In JavaScript..

- ❑ A HTML document is an *object* (**document**)
- ❑ It has *properties*
 - **title**="A new Title"
- ❑ It has *methods*
 - **write**("<h1>My Home Page</h1>")
- ❑ You can link methods with certain *events*
 - **onLoad** and **onClick**

JavaScript Basics



Structure

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<script type="text/javascript">  
    javascript code  
    :  
</script>
```

```
</head>
```

```
<body>
```

```
:
```

```
</body>
```

```
</html>
```

```
<script type="text/javascript">
```

- ☐ Container tag for JavaScript
- ☐ Can be in `<head>` or `<body>`
- ☐ Can have more than one `<script>`

Variables

- ❑ Variables are used to store data.
- ❑ A variable is a **container for information** that you want to store.
- ❑ A variable's **value can change** during the script.
- ❑ You can **refer to a variable by name** to see its value or to change its value.



Variables

- ❑ Variables contain values and use the equal sign (=) to specify their value.
- ❑ Variables are created by declaration using the **var** keyword with or without an initial value state.
 - **var valueA;**
 - **var valueB = 10;**
 - **var valueC = 100;**



Variable Names

- ❑ An *identifier* is a unique name (for a variable, an object, or a function)
- ❑ Variable names are **case-SENSITIVE**
 - **index != INDEX != Index**
- ❑ Must start with **a letter, underscore character or \$**
- ❑ Can contain **letters** (both upper/lowercase) and **digits**
 - Contains only 'A' – 'Z', 'a' – 'z', '0' – '9', '_', '\$'
 - First character **cannot be a digit**
 - **Cannot be reserved words** or keywords

Variable Names

Valid Names

myVariableExample
My_variable
\$my_variable
_my_variable
My_variable_example

- An *identifier* is a unique name (for a variable, an object, or a function)
 - Must start with a **letter** or the **underscore character** or **\$**
 - Can contain **letters** (both upper/lowercase) and **digits**
- Contains only 'A' – 'Z', 'a' – 'z', '0' – '9', '_', '\$'
- First character **cannot be a digit**
- **Cannot be reserved words** or keywords

Invalid Names

1my_example
@my_variable
my variable
%my_variable
#my_variable
~my_variable



Variables and Scope

Scope:

Places in the program where a particular identifier is associated with a particular data store.



Variables and Scope

`var identifier;`

- ❑ The scope of *identifier* is ...
 - The function in which `var identifier` appears = *local*
 - Otherwise, the *entire Web page* = *global*

`function identifier(parameters) { ... }`

- ❑ The scope of the `parameters` is `{ ... }`
 - The scope of `identifier` is the entire Web page

```
// code here can not use carName

function myFunction() {
    var carName = "Volvo"; //local

    // code here can use carName
}
```

```
var carName = " Volvo"; //global

// code here can use carName

function myFunction() {

    // code here can use carName
}
```

Constants

- ❑ Constants were introduced to Javascript only in 2015.
- ❑ Constants are declared with the `const` keyword.
- ❑ In Javascript, `const` does not define a constant value. Rather, it defines a constant reference to a value.
 - ❑ Which is why its called a `const` variable.
- ❑ A `const` variable cannot be reassigned a new value.
- ❑ PHP can also declare global constants with the `define` function – e.g. `define("PI", "3.14159");`

```
const PI = 3.141592653589793;
```

```
// This will give an error
```

```
PI = 3.14;
```

```
// This will also give an error
```

```
PI = PI + 10;
```

Primitive Types

- ❑ Five primitive types
 - Number
 - String
 - Boolean
 - Undefined
 - Null
- ❑ Two other primitive types
 - Symbol (introduced in 2015)
 - BigInt (introduced in 2020)
 - We don't cover these in this course



Primitive Types - Number

- ❑ **integers** – whole-number values

```
var myint = 10;
```

```
var myint2 = 20
```

- ❑ **floating-point** – real numbers; with a fraction

```
var myfloat = 10.6;
```

```
var mybigfloat = 6.023e23;
```

- **e** = *scientific notation*

- $3e5 = 3 \times 10^5 = 3 \times (10 \times 10 \times 10 \times 10 \times 10)$

Primitive Types - Boolean, Null, Undefined

- ❑ **boolean** – can be **true** or **false**
`var myboolean = true;`
`var myotherboolean = false;`
- ❑ **null** – a null value, a reserved word
- ❑ **undefined** – declared but not assigned a value

Primitive Types - Boolean, Null, Undefined

```
<html>
<head>
<title> Null and Undefined example </title>
<script type="text/JavaScript">
    var test1, test2 = null;
</script>
</head>
<body> ... </body>
</html>
```

test1

- test1 is not assigned a value
- Its value is undefined

test2

- test2 is assigned a value
- Its value is null



Primitive Types - Strings

- ❑ A string variable can store a sequence of alphanumeric characters, spaces and special characters.
- ❑ A string can be enclosed by a pair of single quotes (') or double quote (").
- ❑ Use escaped character sequence to represent special character (e.g., \", \n, \t)



Primitive Types - Strings

```
var a_string="This is an example";
```

```
var b_string="";
```

```
var myString = "punk" + "rock"
```

assigns the string **"punkrock"**
to the variable **myString**

```
var myString = "punk" + " " + "rock"
```

assigns the string **"punk rock"**
to the variable **myString**

What would the variable 'x' contain after the execution of the following lines of codes: var a = "2"; var b = "3"; var x = a+b;

2

0%

235

0%

23

0%

3

0%

5

0%





Primitive Types

PHP and JavaScript do not stop you from changing the type of a variable – this is *weak (loose) typing*

```
var myint=20;  
myint="Ok, so now it is a string";  
myint=20e10;  
myint=true;
```

DO NOT DO THIS!

- ❑ Leads to very messy, buggy and hard-to-follow code
- ❑ Treat both PHP and JavaScript as if they were *strongly typed*
 - ❑ After you say `var myint=20`, you should only use `myint` as an integer

Operators

- ❏ Operators are used to handle variables.
- ❏ Types of operators with examples:
 - Arithmetic operators, such as **plus (+)**
 - Comparisons operators, such as **equals (==)**
 - Logical operators, such as **and (&&)**.
 - Control operators, such as **if**.
 - Assignment and String operators.



JavaScript - Arithmetic Operators

+	Addition	<code>x=2; y=2 x+y</code>	4
-	Subtraction	<code>x=5; y=2; x-y</code>	3
*	Multiplication	<code>x=5; y=4; x*7</code>	20

/	Division	<code>15/5 5/2</code>	3 2.5
%	Modulus (remainder division)	<code>5%2 10%8 10%2</code>	1 2 0
++	Increment	<code>x=5; x++</code>	x=6
--	Decrement	<code>x=5; x--</code>	x=4

JavaScript - Arithmetic Operators

❏ Examples

```
var x = 20; var y = 30; var z = 10;
```

```
x = y/(z-20);
```

```
y = (z-9) * x;
```

```
z--; z++; z--;
```

❏ What are the values of **x**, **y** and **z** after each statement?

What would the variable 'x' contain after the execution of the following lines of codes: `var a = 2;`
`var b = a++;` `var c = a;` `var x = ++a + b + c;`

6

0%

7

0%

8

0%

9

0%

10

0%





JavaScript - Comparison Operators

Operator	Description	Example
==	is equal to	5==8 returns false
!=	is not equal	5!=8 returns true
>	is greater than	5>8 returns false
<	is less than	5<8 returns true
>=	is greater than or equal to	5>=8 returns false
<=	is less than or equal to	5<=8 returns true

JavaScript - Comparison Operators

❏ Examples

<code>20 < 30</code>	<code>==>true,</code>
<code>30 == 29</code>	<code>==>false,</code>
<code>20 <= 20</code>	<code>==>true,</code>
<code>8.34 != 1.01</code>	<code>==>true,</code>
<code>(-100 > -190)</code>	<code>==>true,</code>
<code>(10-20)*10 >= 10-(20*10)</code>	<code>==>true</code>



JavaScript - Logical Operators

Operator	Description	Example
&&	and	<code>x=6; y=3;</code> <code>(x < 10 && y > 1)</code> returns true
	or	<code>x=6; y=3;</code> <code>(x==5 y==5)</code> returns false
!	not	<code>x=6; y=3;</code> <code>!(x==y)</code> returns true

**These also work in PHP,
the syntax is identical!**

JavaScript - Logical Operators

❏ Examples

```
var a = true; var b = false; var c = true;
```

```
a && c           ==> true
```

```
a || b           ==> true
```

```
!b               ==> true
```

```
a && (b || b)     ==> false
```

```
!(!a || b) && !b  ==> true
```



JavaScript - Assignment Operators

❏ Examples:

<code>var x = 1; var y = 2; var z = 3;</code>	
<code>x = 10;</code>	x is assigned the value 10
<code>y += x;</code>	y is assigned the value 12 (2+10)
<code>z -= x;</code>	z is assigned the value -7 (3-10)
<code>x *= y;</code>	x is assigned the value 120 (10*12)
<code>x /= 12;</code>	x is assigned the value 10 (120/12)

❏ PHP also supports these augmented assignment operators.



Popup box - alert Method

- **alert** is a built-in function
- it creates a pop-up dialog displaying
 - a piece of text and
 - an OK/Close button (which closes the dialog) - depending on the OS/Browser

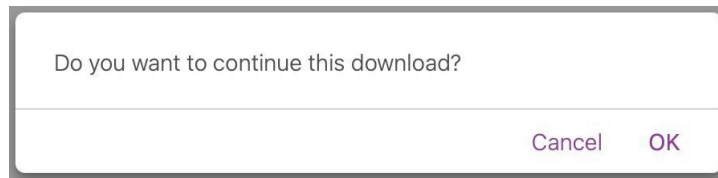
```
alert("Form Completed");
```



Popup box - confirm Method

- ❑ The confirm method displays a message provided as a parameter
 - The confirm dialog has two buttons: OK and Cancel
- ❑ If the user presses OK, **true** is returned by the method
- ❑ If the user presses Cancel, **false** is returned

```
var question = confirm("Do you want to  
continue this download?");
```



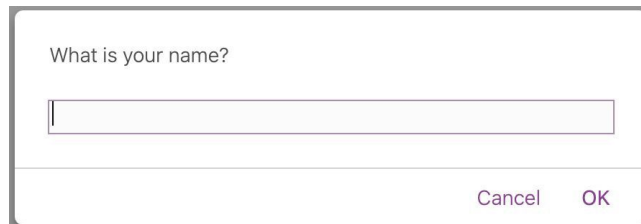


Popup box - prompt Method

- ❑ This method displays its string argument in a dialog box
 - A second argument provides a default content for the user entry area
- ❑ The dialog box has an area for the user to enter text
- ❑ The method returns a **String** with the text entered by the user
- ❑ You can also provide a **default value for the prompt.**

```
name = prompt("What is your name?", "Don");
```

```
name = prompt("What is your name?", "");
```



The First Script

```
<!DOCTYPE html>
<html>
  <head>
    <script type="text/javascript">
      alert("Hello world");
    </script>
  </head>
  <body>
    :
    :
  </body>
</html>
```



Control Statement - Decisions

```
if (condition)  
    {statements}
```

- The *condition* between the **()** is evaluated
- If *condition* is true, then the *statements* are executed
- If *condition* is false, then the *statements* are skipped

```
if (condition)  
    {statementsA}  
else  
    {statementsB}
```

- If *condition* is true, then the *statementsA* are executed
- If *condition* is false, then the *statementsB* are executed

Control Statement - Decisions

❏ Example

```
if (warning_level <= 50)
    { document.body.style.color = 'black'; }
else
    { document.body.style.color = 'red'; }
```



Control Statement - Looping

```
while (condition)  
    { statements }
```

- ❏ The *condition* between the **()** is evaluated
 - If *condition* is true, then the *statements* are executed, and we repeat from 1
 - If *condition* is false, then the *statements* are skipped and the while loop ends

Control Statement - Looping

❏ Example

```
var i=1;
while (confirm("At " + i + " - keep going?"))
    { i++; }
```

- ❏ The confirm methods displays a message provided as a parameter
 - The confirm dialog has two buttons: OK and Cancel
- ❏ If the user presses OK, true is returned by the method
- ❏ If the user presses Cancel, false is returned

Control Statement - Looping

```
<body>
  <p>
    <script type="text/javascript">
      var counter = 100;
      var numberlist = "";
      while (counter > 0) {
        numberlist += "Number " + counter + "<br>";
        counter -= 10;
      }
      document.write(numberlist);
    </script>
  </p>
</body>
```

Number 100
Number 90
Number 80
Number 70
Number 60
Number 50
Number 40
Number 30
Number 20
Number 10

What will be last value at the console output after the execution of the code: `var counter =10; while(counter-- > 0) {console.log(counter);}`

9

0%

1

0%

0

0%

-1

0%

10

0%





Control Statement - Looping

```
for (initial; condition; update)  
    {statements}
```

1. The **first time** this **for** loop is encountered, the *initial* expression is evaluated
2. Then, the *condition* is evaluated
 1. If *condition* is true, then the *statements* are executed, the *update* expression is evaluated, and repeat from 2
 2. If *condition* is false, then the for loop ends – the *statements* and the *update* expression are skipped
 3. The *condition* describes the stopping point for the loop

Control Statement - Looping

❏ Example

```
for (var i=1; i<=8; i++) {  
    document.write('<p>the value of i is ' + i + '</p>')  
}
```



Functions

- ❑ You can define your own functions.
- ❑ Code structure –
 - splitting code into parts (functions)
- ❑ Functions are like **mini-programs**.

```
function <function_name>  
(parameters)  
{  
    // code segments;  
}
```



Functions

- ❏ Data comes in, processed, result returned

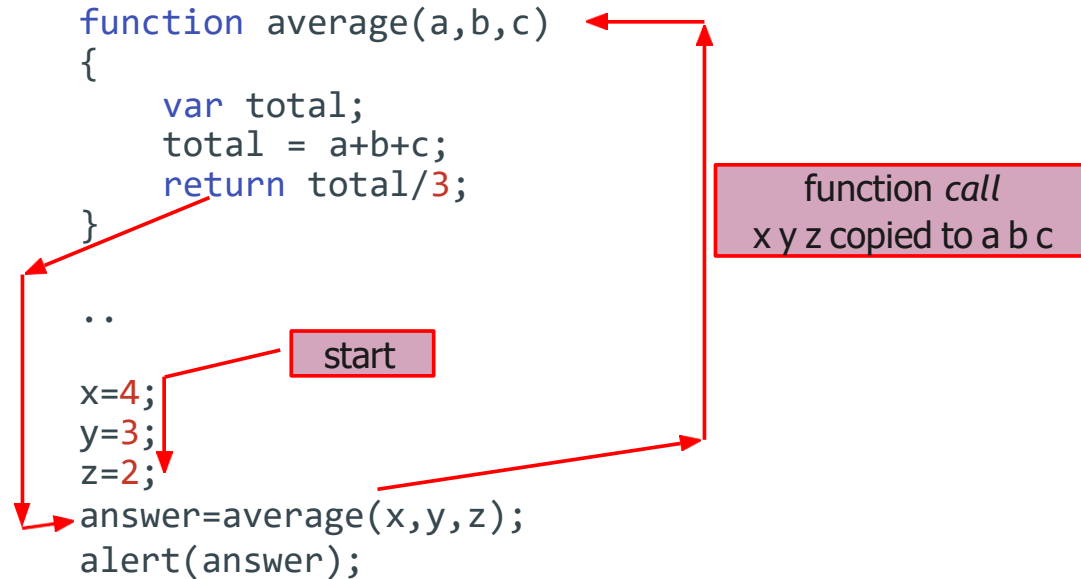
```
function average(a,b,c)
{
    var total;
    total = a+b+c;
    return total/3;
}
```

Diagram illustrating the function `average(a,b,c)`:

- The parameters `a, b, c` are highlighted with an orange box, and an arrow points to a purple box labeled "Values come in here".
- The return statement `return total/3;` is highlighted with an orange box, and an arrow points to a purple box labeled "Value returned here".

- ❏ **Note:** Parameters are optional. A function doesn't always need to have parameters.

Functions





Functions (Return Values)

- ❑ A function can return value of any type using the keyword "return".
- ❑ The same function can possibly return values of different types
- ❑ If no value is being explicitly returned **"undefined"** is returned.

```
function foo (p1) {  
    if (typeof(p1) == "number")  
        // Return a number  
        return 0;  
  
    else if (typeof(p1) == "string")  
        // Return a string  
        return "zero";  
}
```

`typeof` is a special function that returns the type of a value.

Functions (Return Values)

```
function foo (p1) {  
  if (typeof(p1) == "number")  
    // Return a number  
    return 0;  
  
  else if (typeof(p1) == "string")  
    // Return a string  
    return "zero";  
  
  // If no value being explicitly returned  
  // "undefined" is returned.  
}
```

```
foo(1);           // returns 0  
foo("abc");       // returns "zero"  
foo();            // returns undefined
```



JavaScript - onclick

`onclick="function_name(parameters)"`

❏ `onclick` is an **event**

- The JavaScript code in the attribute value is executed when the user clicks on the button

```
<input type="button" value="red"
      onclick="change_colour('red')">
<input type="button" value="green"
      onclick="change_colour('green')">
```

JavaScript - Example 1

```
<html>
  <head>
    <script type="text/javascript">
      function change_colour(new_colour)
      {
        document.body.style.backgroundColor = new_colour;
      }
    </script>
  </head>
  <body>
    <form>
      <input type="button" value="red"
        onclick="change_colour('red')" />
      <input type="button" value="green"
        onclick="change_colour('green')" />
      <input type="button" value="purple"
        onclick="change_colour('purple')" />

    </form>
  </body>
</html>
```

Document Object Model

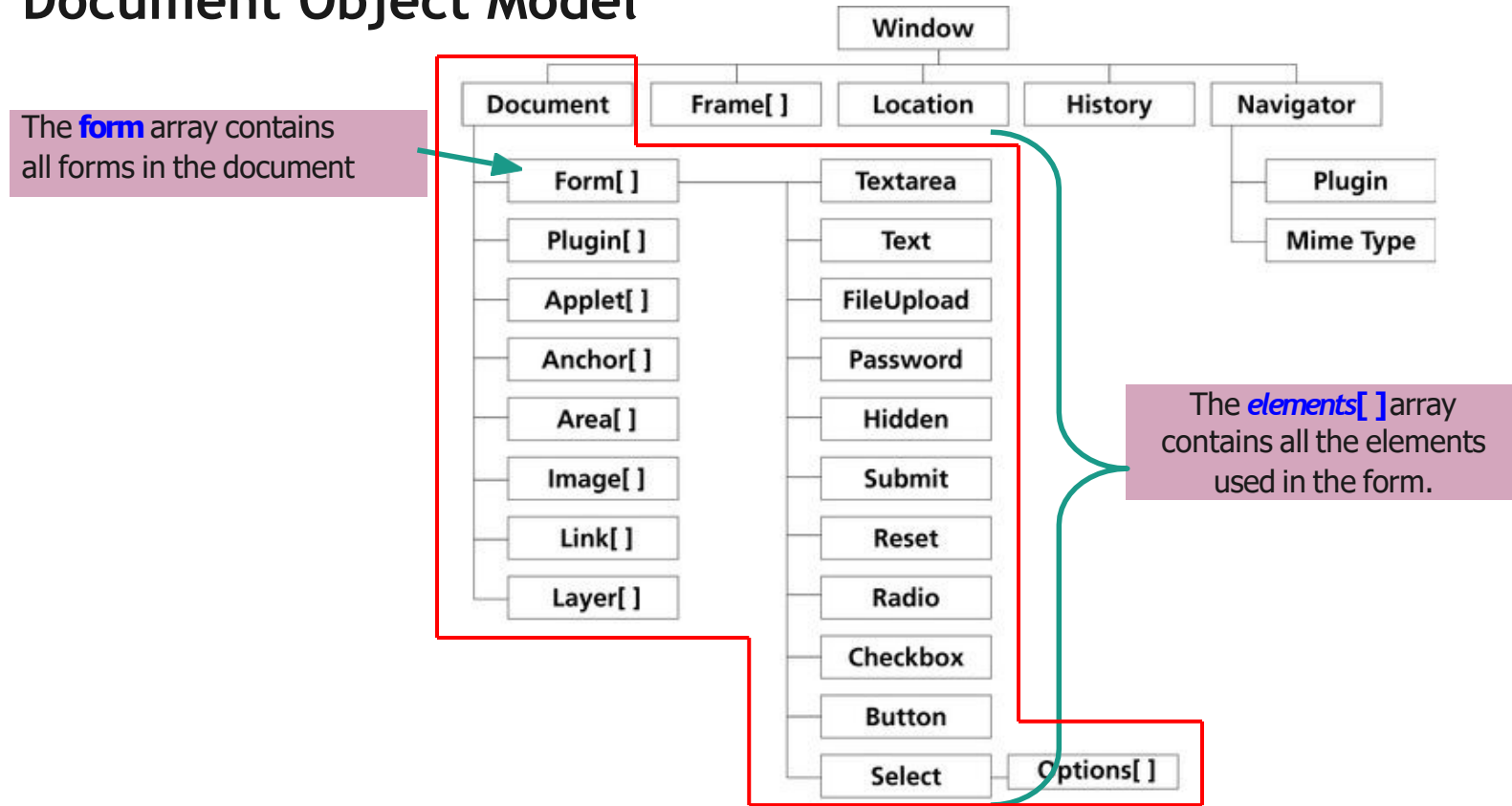


Figure 5-2: Browser object model

DOM: Document Object Model

❏ The window object

- The highest-level object
- It is the default object
- `window.document.write("a test message");`
- We may omit writing window explicitly
 - ❏ `document.write("a test message");`

❏ The document object

- A web document or a page in a browser window

❏ The form object

- The browser creates a unique "form" object for each form in a document
- `document.formname` or `document.forms[1]`



Document Object Model

`document.formname.elementname.value`

```
<html>
:
<body>
  <form name="myform">
    <label for="id_name">Name</label>
    <input type="text" name="name_name" id="id_name">
    <br><br>

    <label for="id_email">Email</label>
    <input type="email" name="name_email" id="id_email">
  </form>
</body>
<script>
  document.myform.name_name.value = 'SENG1050'
</script>
</html>
```

Examples

`document.myform.name_name.value`
`document.myform.name_email.value`

- ❑ **value** is a property of Textbox.
- ❑ Using javascript you can:
 - access what's typed in a textbox
 - Or change the value

Name

Email



Document Object Model

`document.getElementById(element_id)`

```
<html>
  :
  <body>
    <form name="myform">
      <label for="id_name">Name</label>
      <input type="text" name="name_name" id="id_name">
      <br><br>

      <label for="id_email">Email</label>
      <input type="email" name="name_email" id="id_email">
    </form>
  </body>
  <script>
    document.getElementById('id_email').value = 'SENG1050@uon.edu.au'
  </script>
</html>
```

Examples

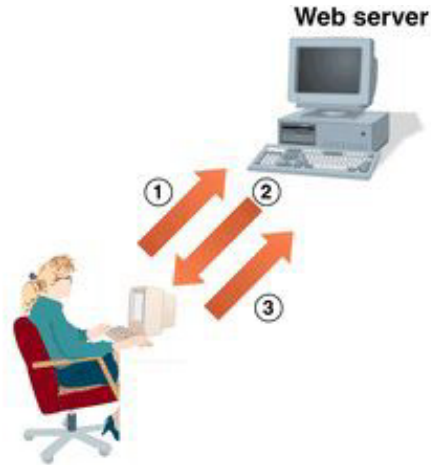
```
document.getElementById('id_name')
document.getElementById('id_email')
```

Form Validation

Form Validation

- ❑ Before an HTML form is submitted, JavaScript can be used to provide client-side data validation.
- ❑ If the form is not valid, the form is not submitted until the errors are fixed
- ❑ For example,
 - were required fields left empty?
 - was a valid e-mail address entered?
 - was a valid date entered?
 - was text entered in a numeric field?
 - were numbers entered in an text field?
 - did the number entered have a correct range?

Form Validation



Server-side validation

- 1) The user submits the form to the Web server.
- 2) The Web server validates the user's responses and, if necessary, returns the form to the user for correction.
- 3) After correcting any errors, the user resubmits the form to the Web server for another validation.



Client-side validation

- 1) The user submits the form, and validation is performed on the user's computer.
- 2) After correcting any errors, the user submits the form to the Web server.

Form Validation - Text Box

- ❏ Let's validate that the name field in our form is not blank.

```
<form name="easyform" ... >
  Please enter your name <input type="text" name="yourname">
  Please enter your birth year <input type="text" size="5" name="birthyear">
  ... other inputs here
</form>
```

- ❏ The contents of these two fields are
 `document.easyform.yourname.value`
 `document.easyform.birthyear.value`

Form Validation - Text Box

❏ To validate the name field

if (the name text box is blank){

Show an alert with a reminder message

Form is not done yet

} else {

Show a personalized thank you message

Form is done

}

`document.easyform.yourname.value == ""`

`alert("argh! Please provide your name.");`

`return false;`

`alert("Thank you," +
document.easyform.yourname.value+".");`

`return true;`

Form Validation - Text Box

❏ Javascript

```
<head>
<script type="text/javascript">
    function checkName()
    {
        if (document.easyform.yourname.value == "") {
            alert("argh! Please provide your name.");
            return false;
        } else {
            alert("Thank you, " + document.easyform.yourname.value + ".");
            return true;
        }
    }
    /* checkName() */
</script>
</head>
```

`checkName()` is a boolean function that will

1. Return true if the name field is filled out, and
2. Return false if it is blank



Form Validation - Text Box

To call `checkName()`, you can specify this either:

1. in the submit button's `onClick` handler, or
2. in the form's `onSubmit` handler.

But, not in both!

Form Validation - Text Box

```
<head>
<script type="text/javascript">
    function checkName()
    {
        if (document.easyform.yourname.value == "") {
            alert("argh! Please provide your name.");
            return false;
        } else {
            alert("Thank you, " + document.easyform.yourname.value + ".");
            return true;
        }
    }
</script></head>
<body>
    <form name="easyform">
        <label for="yourname">Name</label>
        <input name="yourname"><br><br>
        <input type="submit" value="Submit" onClick="return checkName();">
    </form>
</body>
</html>
```

In the submit button's `onClick` handler

Form Validation - Text Box

```
<head><script type="text/javascript">
  function checkName()
  {
    if (document.easyform.yourname.value == "") {
      alert("argh! Please provide your name.");
      return false;
    } else {
      alert("Thank you, " + document.easyform.yourname.value + ".");
      return true;
    }
  }
  /* checkName() */
</script></head>
<body>
  <form name="easyform" onSubmit="return checkName()">
    <label for="yourname">Name</label>
    <input name="yourname"><br><br>
    <input type="submit" value="Submit">
  </form>
</body>
```

In the form's `onSubmit` handler

Form Validation - Checkboxes

```
<form name="myform">
  Were you a ghost for halloween?<br>
  <input type="checkbox" name="ghost" value="YES" id="yes">
  <label for="yes">Yes</label><br>

  <input type="checkbox" name="ghost" value="NO" id="no">
  <label for="no">No</label><br>

  <input type="checkbox" name="ghost" value="Secret" >
  <label for="secret">Secret</label><br>
</form>
```

- ❑ How are the checkboxes accessed?
 - By their name
- ❑ How are checkboxes in a group accessed.
 - By using index numbers
 - Index numbers start at 0

`document.myform.ghost[0]` => "Yes" box

`document.myform.ghost[1]` => "No" box

`document.myform.ghost[2]` => "Secret" box



Form Validation - Checkboxes

- ❑ The **checked** property is true when the box has a check in it, and it's false otherwise.

```
if ( the first check box is not checked AND  
    the second check box is not checked AND  
    the third check box is not checked ) {  
    Show alert message  
    Not done yet  
}
```

```
if ( !document.myform.ghost[0].checked &&  
    !document.myform.ghost[1].checked &&  
    !document.myform.ghost[2].checked ) {  
    alert("Please answer the ghost question.");  
    return false;  
}
```



Form Validation - Radio Buttons

- ❑ How are the radio buttons named?
- ❑ By counting them.

```
<form name="myform">
  <input type="radio" name="urgency" value="high" id="high">
  <label for="high">High</label><br>
  <input type="radio" name="urgency" value="medium" id="medium">
  <label for="medium">Medium</label><br>
  <input type="radio" name="urgency" value="low" id="low">
  <label for="low">Low</label>
</form>
```

```
if ( !document.myform.urgency[0].checked &&
    !document.myform.urgency[1].checked &&
    !document.myform.urgency[2].checked &&) {

    alert("Please select the urgency.");
    return false;
} else {
    return true;
}
```



Form validation - pulldown menu

- ❑ **selectedIndex**: it's value is the number (starting, of course, from 0) of the menu item that was selected.

```
if (document.easyform.cartoon.selectedIndex == 0) {  
    alert("                ");  
    return _____;  
} else {  
    alert("  
    return _____;  
}
```

Document.easyform.cartoon.selectedIndex	Menu choice
0	Select one
1	Bugs Bunny
2	Mickey Mouse
3	Simpsons
4	Futurama

Which of the following lines of JavaScript would get the value of a textbox with the name attribute 'fname' and assign it to the variable 'myname'?

```
var myname = document.fname;
```

0%

```
var myname = document.fname.value;
```

0%

```
var myname = document.getElementById("fname").value;
```

0%

```
var myname = document.get("fname");
```

0%



Comments and Methods



Comments

- ❏ You MUST comment your JavaScript
 - Good coding practice
 - It makes it easier to understand
 - Useful if you want to reuse your code later
 - Useful if someone else needs to understand or reuse your code
 - Needed to get full marks in assignments



Comments

❏ `// a comment line`

- Everything to the end of the line is a comment

❏ `/* a comment block */`

- Everything inside the block is a comment – even over multiple lines

❏ `<!-- a comment line`

- The start of an HTML comment also acts as a JavaScript line comment
- Placing `<!--` at the start of a script and `// -->` at the end hides the script from old browsers that don't understand it

```
<script language="JavaScript">  
<!--  
Here you put your JS code.  
Old browsers will treat it as an HTML comment.  
//-->  
</script>
```

Methods

- ❑ [Date Methods](#) - set variables to the clock time in a variety of ways.
- ❑ [Window Methods](#) - used to open and close new windows
- ❑ [Document Methods](#) - Generate new documents on the fly.
- ❑ [Form Methods](#) - select form items, send the cursor to text boxes and submit forms.
- ❑ [History Methods](#) - Press the reader's 'Back' button and other tricks.
- ❑ [Math Methods](#) - sin, cos, round, random, absolute value, etc.
- ❑ [MessageBox Methods](#) - Alert, Prompt, and Confirm are all methods.



Date Methods

```
var x = new Date();  
x.setFullYear(2100,0,14);  
var today = new Date();
```

`new Date()`: returns the current date

```
if (x < today) {  
    alert("Today is before 14th January 2100");  
}  
else {  
    alert("Today is after 14th January 2100");  
}
```

We can compare dates; today is less than tomorrow

Window Methods

- ❑ Open a blank page in a new window/tab & focus on it

```
var myWindow = window.open("", "", "width=200,height=100");  
myWindow.focus()
```

- ❑ Open a new window/tab and write some text into it

```
var myWindow = window.open("", "MsgWindow", "width=200,height=100");  
myWindow.document.write("<p>This is 'MsgWindow'. I am 200px wide and 100px tall!</p>");
```

- ❑ Open "<https://www.newcastle.edu.au/>" in a new window/tab

```
window.open("https://www.newcastle.edu.au/");
```

Math Methods

- ❑ The Math object has various properties and methods.

- ❑ You access properties with the syntax

`Math.property`

`Math.PI` // returns PI

- ❑ You call Math methods with `Math.method(number)`

`Math.round(x)` returns the nearest integer

`Math.round(4.9);` // returns 5

`Math.round(4.2);` // returns 4

`Math.pow(x, y)` returns the value of x^y

`Math.pow(8, 2);` // returns 64

`Math.sqrt(x)` returns the square root of x

`Math.sqrt(64);` // returns 8



Document Methods

- ❑ The document object represents your web page
- ❑ You can write some text directly to the document

```
var myWindow = window.open("", "MsgWindow", "width=200,height=100");  
myWindow.document.write("<p>This is 'MsgWindow'. I am 200px wide and 100px tall!</p>");
```

❖ *Here, we've first created a new window, and then written something to the document in that window.*

- ❑ If you want to access any element in an HTML page, you always start with accessing the document object.

```
document.getElementById('id_email').value = 'SENG1050@uon.edu.au'
```

- ❑ Elements can be removed with the remove() method. e.g.
`document.getElementById('id_email').remove()`



Document Methods

- ❑ Creating new elements in the DOM involves two steps: creating the element, and adding it to the DOM.
- ❑ To create an element, use the function `createElement(type)` or `createTextNode(value)`
- ❑ Once created, the element can be added to the DOM by accessing an existing element and using the function `insertAdjacentElement(position, insertElement)`
- ❑ The values for 'position' are:
 - afterbegin (As the first child of the element)
 - afterend (Directly after the element)
 - beforebegin (Directly before the element)
 - beforeend (As the last child of the element) – (you can also use the `appendChild()` function for this)

```
// Create a paragraph node and some content for it
const newNode = document.createElement("p");
const content = document.createTextNode("Hello World");
```

```
// Add the content to the paragraph node
// newNode.insertAdjacentText("afterbegin", content);
newNode.appendChild(content);
```

```
// Add the new paragraph node to the DOM
const target = document.getElementById("insert");
target.insertAdjacentElement("beforeend", newNode);
```



Document Methods

- ❑ The document object can also be used to write and read cookies.
- ❑ A cookie is a small text file stored on the client that is used to store data (name/value pairs e.g. "username=Bob;").
- ❑ Often used for login tokens, trackers, user profile data, and much more.
- ❑ Privacy implications – some users do not want to be tracked!
- ❑ Use document.cookie to read AND write cookies

```
document.cookie = "username=Bob";  
document.cookie = "favourite=Pizza"; // Write a cookie  
var cookies = document.cookie; // Read all cookies
```

History Methods

- ❑ The `window.history` object contains the browser's history.
- ❑ But, to protect the privacy of the users, there are limitations to how JavaScript can access this object.
- ❑ Some methods:
 - `history.back()` - same as clicking back in the browser
 - `history.forward()` - same as clicking forward in the browser

```
<html>
  <head>
    <title>History</title>
    <meta charset="utf-8">
    <script>
      function goBack()
      {
        window.history.back()
      }
    </script>
  </head>
  <body>
    <button onClick="goBack()">Go
    Back</button>
  </body>
</html>
```

Loading URL With JavaScript

❏ `location.href = url;`

- **location** is a different object to **document**
- It has a property **href**
- In this case, **url** (which is either ["http://www.yahoo.com/"](http://www.yahoo.com/) or ["http://www.google.com/"](http://www.google.com/)) is assigned to **href**
- This causes the browser to immediately load this new location

```
:
<script type="text/javascript">
  function jump() {
    var select = document.getElementById("jumpTo");
    var index = select.selectedIndex;
    var url = select.options[index].value;
    location.href = url;
  }
</script>

:
<form>
  <select id="jumpTo" size="1">
    <option value="http://www.yahoo.com/">Yahoo</option>
    <option
      value="http://www.google.com/">Google</option>
  </select>
  <input type="button" value="Go!" onclick="jump()">
</form>
```



Summary

- ❑ We have completed the separation of concerns, now our data, structure, styling, and behaviour are all separated from each other on the client-side and the server-side, so they can be varied independently without affecting other components.
- ❑ We have looked at how to use JavaScript to implement client-side behaviour, to validate forms, write content to the DOM, and use cookies.
- ❑ The next step is to make our website visually appealing, accessible, responsive, fast, and optimised for search engines. We will consider responsive, fast web design, using media queries, principles of human-computer interaction (HCI), and basic search engine optimisation (SEO) – next week.

References

- ❑ Web Development and Design Foundations with XHTML Terry Felke Morris, AW 2009, Chapter 14
- ❑ Doing Web Development: Client-Side Techniques by Deborah Kurata, APress, 2002 – Chapters 7-10
- ❑ David Flanagan: JavaScript: The Definite Guide. O'Reilly, 6th edition, 2011.
- ❑ Michael Bolin: Closure: The Definite Guide. O'Reilly, 2010.
- ❑ JavaScript Tutorials
 - <http://www.webteacher.com/javascript/index.html>
 - <http://www.pageresource.com/jscript/index2.htm>
 - <http://javascript.internet.com/>
 - <http://www.w3schools.com/js/default.asp>